GUI Calculator - Project Report

Tal Fiskus (208423707)

March 18, 2026

1 Introduction

The objective of this project is to develop a robust and intuitive desktop-based graphical calculator application. The calculator caters to both basic arithmetic needs and common scientific calculations, providing a seamless user experience through a modern, dark-themed interface. This project was developed as part of the **Reinforcement Learning (RL) Course** at **Bar Ilan University (BIU)**.

2 Functional Requirements

2.1 User Interface (UI)

- **Framework:** Developed using Python's `tkinter`.
- **Display:** A primary digital display area for current inputs and real-time calculation results.
- **Layout:** A grid-based layout for digits (0-9) and operation buttons.
- **Theme:** A sleek, dark-themed UI for reduced eye strain and modern aesthetics.

2.2 Core Operations

- **Basic Arithmetic:** Addition (+), Subtraction (-), Multiplication (*), and Division (/).
- **Advanced Math:** Square Root ($\sqrt{x}$), Exponentiation (x^y), and Reciprocal ($1/x$).
- **Precision:** Support for high-precision floating-point calculations.

2.3 Memory and History

- **History Log:** A dedicated panel that tracks and displays a scrollable list of the last 10 calculations, allowing users to reference previous results.

2.4 Error Handling

- **Graceful Failures:** The application does not crash on invalid inputs.
- **Messaging:** Clear on-screen alerts for mathematical errors, such as *"Error: Div by 0"* or *"Error: Invalid Input"*.

3 Technical Specifications

- **Language:** Python 3.x
- **GUI Library:** Tkinter (Standard Library)
- **Platform:** Cross-platform (Windows, Linux, Mac OS)

4 Project Structure

The project consists of the following key files:

- `calculator.py`: The main entry point of the application, handling the Tkinter GUI and user interactions.
- `logic.py`: Contains the mathematical engine and the history management logic, separating calculation logic from the UI.
- `test_logic.py`: A dedicated testing suite for verifying arithmetic correctness, advanced functions, and error resilience.
- `PRD.md`: Product Requirements Document detailing the project goals and specs.
- `README.md`: Comprehensive documentation including installation and user manual.

5 Testing and Validation

The project includes a robust testing suite in `test_logic.py`. These tests ensure:

1. **Arithmetic Correctness:** Verification of results for basic operations.
2. **Advanced Math:** Validation of power (x^y) and square root ($\sqrt{x}$) operations.
3. **Error Resilience:** Checking division by zero and invalid input handling.
4. **History Persistence:** Ensuring that calculations are logged correctly.

6 User Manual

- **Basic Calculation:** Click digits and operators and press `=` or `Enter`.
- **Advanced Buttons:**
  - `x²`: Squares the current number.
  - `√`: Calculates the square root.
  - `^`: General exponentiation.
  - `1/x`: Calculates the reciprocal.
- **Correction:** Use `DEL` to remove the last character or `C` to clear the display.

7 Conclusion

The GUI Calculator successfully meets all functional and non-functional requirements. It provides a reliable tool for basic and scientific calculations with a modern user interface and a robust mathematical core.